

Load Balancing

How a load balancer works, the algorithms that power it, and how to use it — from the basics all the way to production-grade design.

Round Robin

Least Connections

IP Hash

Health Checks

L4 vs L7

High Availability

Builds directly on the previous lesson: **Horizontal vs Vertical Scaling**. Once you have many servers, something has to decide who handles each request. That something is the load balancer.

What's inside

- 1 Why do we even need a load balancer?
- 2 What a load balancer is— and where it sits
- 3 How a single request flows through it (step by step)
- 4 The algorithms — Round Robin, Least Connections, IP Hash (+ two bonuses)
- 5 Health Checks — keeping dead servers out of rotation
- 6 Sticky sessions & the stateless-server upgrade
- 7 L4 vs L7 load balancing (going deeper)
- 8 Don't let the load balancer be the single point of failure
- 9 Advanced extras — SSL termination & global load balancing
- 10 Real-world tools you'll actually use
- 11 Decision guide: which algorithm, when
- 12 Cheat sheet + practice questions

1 Why do we even need a load balancer?

In the last lesson you scaled *horizontally* — instead of one super-powerful server, you ran many ordinary servers side by side. That solved the capacity problem, but it created a new one.

If ten thousand users are hitting your app and you have five identical servers, **who decides which server answers each request?** If everyone naturally lands on Server 1, you're back to a single overloaded machine while the other four sit idle. Horizontal scaling only pays off if the work is actually *spread out*.

CORE IDEA

A **load balancer** is the traffic director that sits in front of your servers and distributes incoming requests across them, so no single server gets overwhelmed while others stay idle.

EVERYDAY ANALOGY

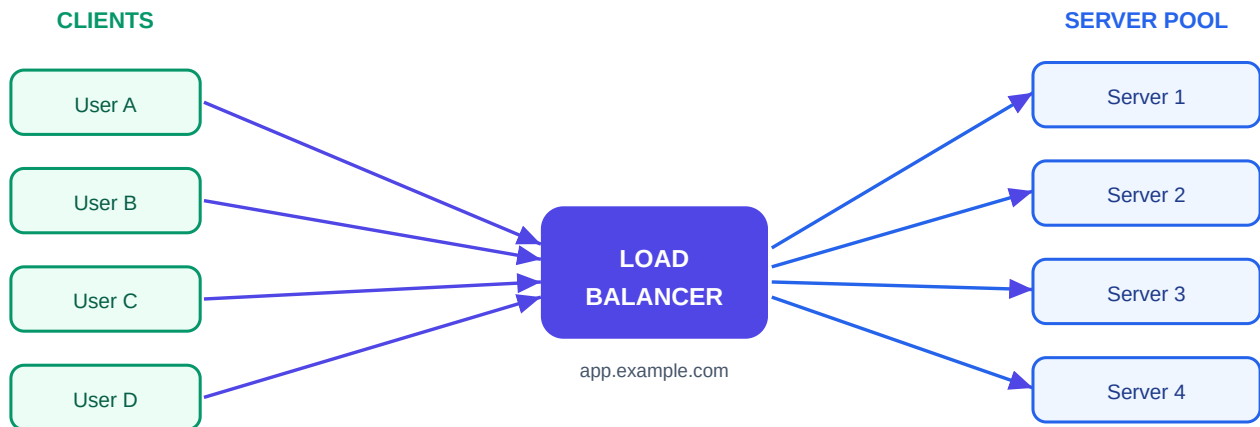
Think of a busy airport check-in hall. Passengers (requests) pour in through the doors. A staff member at the entrance (the load balancer) glances at the counters and says "*you go to counter 3, you go to counter 7.*" Without that person, everyone crowds the first counter they see. With them, all counters stay evenly busy and the queue moves fast.

So a load balancer buys you three things at once:

- **Distribution** — spreads load so each server does a fair share.
- **Availability** — if a server dies, the balancer stops sending it traffic; users never notice.
- **Scalability** — you can add or remove servers behind it without users ever changing the address they connect to.

2 What a load balancer is — and where it sits

Every user connects to *one* public address (say `app.example.com`). That address points at the load balancer, not at any individual server. The real servers live behind it in a private pool and never talk to the public directly.



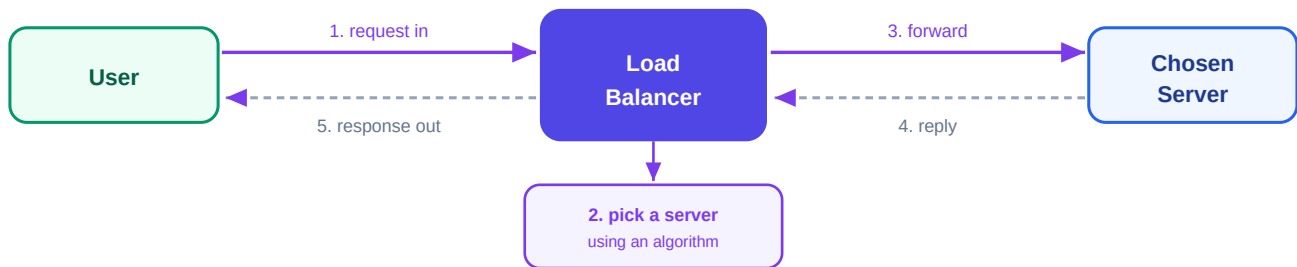
Clients only ever see the load balancer's address. The servers behind it can be added, removed, or replaced freely.

WHY THIS MATTERS

Because clients only know the balancer's address, you can quietly grow from 4 servers to 40 (or shrink back at 3 a.m. to save money) and no user ever has to change anything. This indirection is what makes horizontal scaling practical.

3 How a single request flows through it

Let's follow one request from the moment a user clicks a button to the moment they see a response. This is the loop that repeats millions of times a second.



The magic is entirely in step 2 — choosing which healthy server should handle this request. That choice is the "algorithm."

In words:

- **1. Request arrives** at the load balancer's public address.
- **2. The balancer picks a server** from its pool of currently-healthy servers, using a *load-balancing algorithm* (Section 4).
- **3. It forwards** the request to that server.
- **4. The server processes** it and sends the reply back to the balancer.
- **5. The balancer returns** the response to the user.

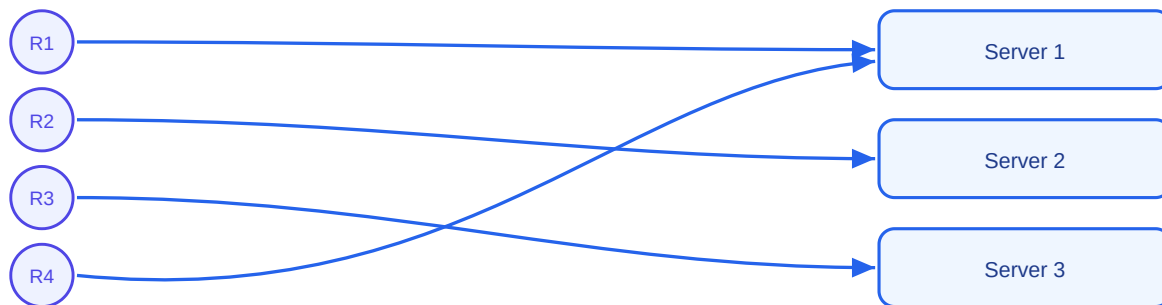
Everything else in this guide is really just *details about step 2* (how to choose) and step 0 (how the balancer knows which servers are healthy in the first place).

4 The algorithms — how it chooses a server

An algorithm is just the rule the balancer follows when picking. Different rules suit different situations. Here are the three you named, done thoroughly, plus two bonuses worth knowing.

4.1 Round Robin simplest

Hand requests out one at a time, in a fixed repeating cycle: Server 1, then 2, then 3, back to 1, and so on — like dealing cards around a table.



R4 wraps back around to Server 1 — the cycle repeats forever

Round Robin: $R1 \rightarrow S1$, $R2 \rightarrow S2$, $R3 \rightarrow S3$, then $R4 \rightarrow S1$ again.

Good because

- Dead simple, no bookkeeping.
- Perfectly even *if* servers are identical and requests cost the same.

Weak because

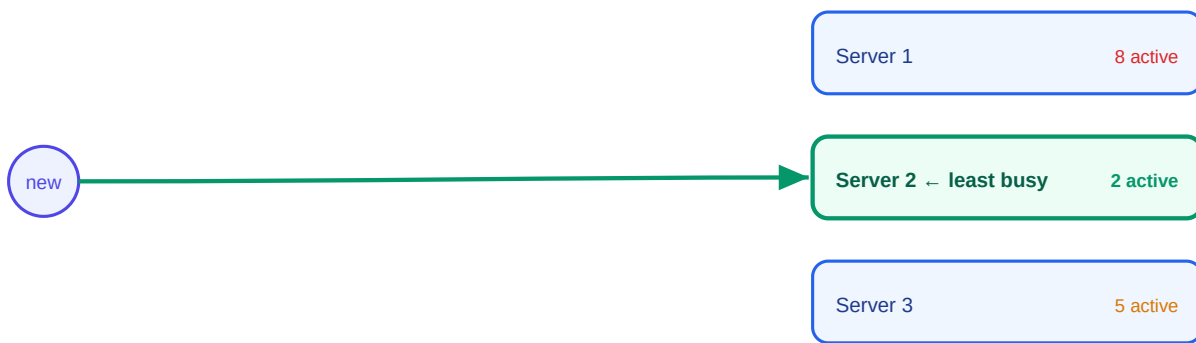
- Ignores how busy each server actually is.
- One slow "heavy" request can pile up on a server that's still "next in line."

WEIGHTED ROUND ROBIN

If some servers are more powerful, give them a **weight**. A server with weight 3 gets three requests for every one that a weight-1 server gets — Round Robin, but the strong servers get dealt more cards.

4.2 Least Connections smartest for uneven work

Instead of blindly cycling, the balancer looks at how many requests each server is *currently* handling and sends the new one to whoever is **least busy** right now.



Server 2 has the fewest active connections (2), so the new request goes there — even though a plain cycle might have picked Server 1.

Good because

- Adapts to reality — heavy/slow requests naturally steer new traffic elsewhere.
- Great when request durations vary a lot (uploads, reports, long queries).

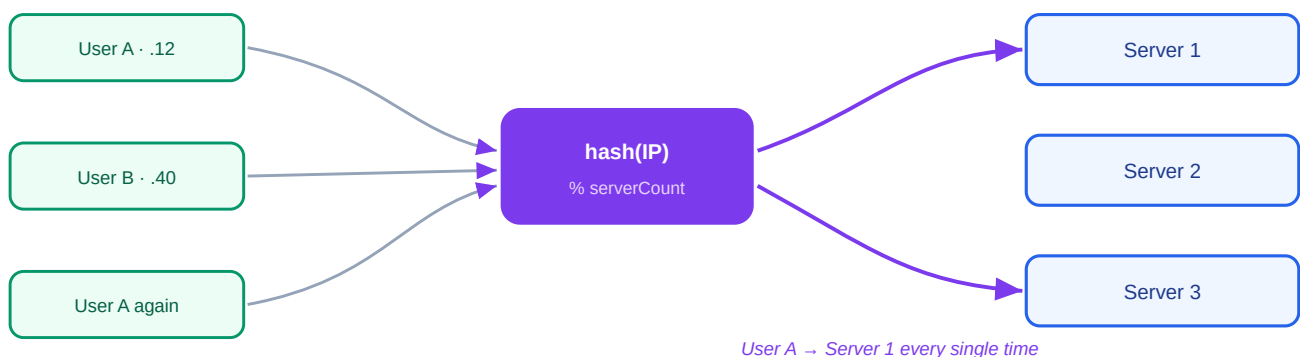
Weak because

- Balancer must track live connection counts (a little more work).
- "Fewest connections" isn't always "least loaded" if requests differ wildly in cost.

There's also **Weighted Least Connections** (factor in server power) and **Least Response Time** (pick the server that's both least busy *and* answering fastest) — refinements of the same idea.

4.3 IP Hash sticky by user

Take the user's IP address, run it through a hash function to get a number, and use that number to pick a server. The same IP always hashes to the same value — so **the same user is consistently sent to the same server**.



Same IP → same hash → same server. User A always lands on Server 1, User B always on Server 3.

WHY YOU'D WANT THIS

If a server stores **session data locally** — your login, your shopping cart, your progress — then you *must* keep sending that user back to the same server, or their cart vanishes. IP Hash guarantees that stickiness for free. (Section 6 shows a cleaner modern alternative.)

Good because

- Same user → same server, so local session data stays valid.
- No extra memory needed on the balancer.

Weak because

- Uneven load if one IP (e.g. a big office/NAT) sends huge traffic.
- Adding/removing a server re-shuffles the mapping and can break sessions.

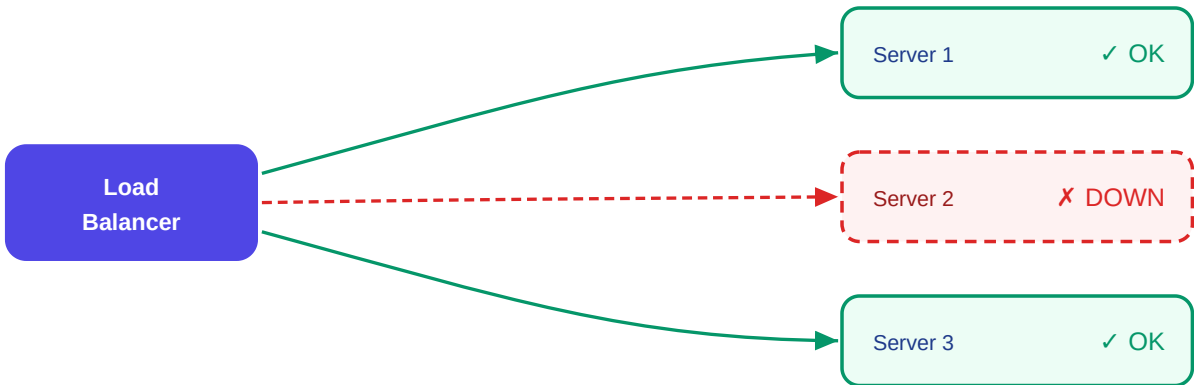
4.4 Two bonus algorithms worth knowing

Algorithm	How it chooses	Use it when
Least Response Time	Picks the server with the lowest combination of active connections <i>and</i> fastest recent response.	Latency matters more than raw evenness.
Random (+ Two Choices)	Pick at random; the popular "power of two choices" variant picks two at random and sends to the lighter of the two.	Huge fleets where tracking every server is expensive — surprisingly even in practice.

5 Health Checks — keeping dead servers out of rotation

None of the algorithms above are safe if the balancer might hand a request to a server that has crashed. So before choosing, the balancer needs to know *which servers are even alive*. That's the job of **health checks**.

DEFINITION
The load balancer regularly contacts each server (e.g. requests `/health` every few seconds). A healthy server replies "OK". If a server fails to reply a few times in a row, the balancer **removes it from rotation** automatically — and quietly adds it back once it recovers.



Server 2 stopped replying → pulled from rotation. No user request ever reaches it.

Health checks run constantly in the background, independent of user traffic.

Two flavours

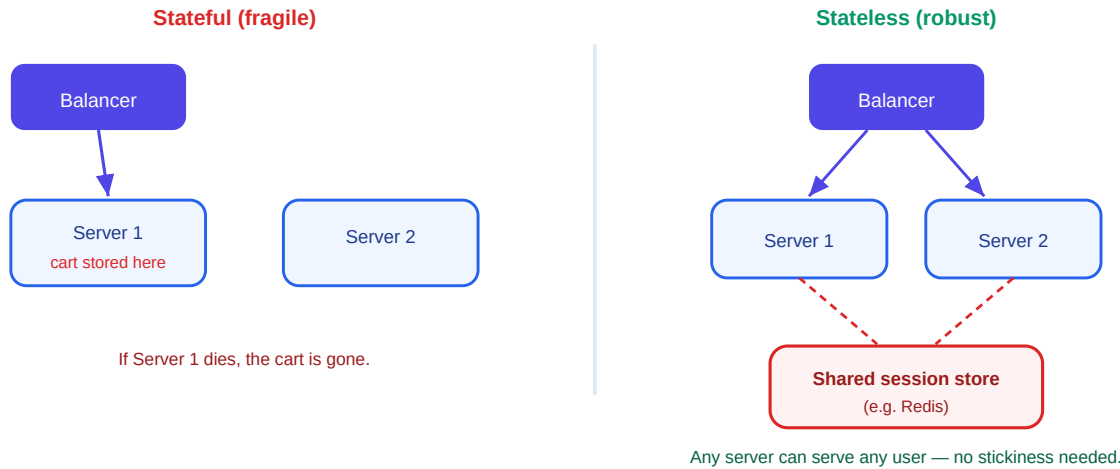
Type	How it works	Trade-off
Active	Balancer proactively pings <code>/health</code> on a timer, whether or not users are around.	Detects problems fast; adds a little background traffic.
Passive	Balancer watches real user requests; if a server starts erroring or timing out, it's marked unhealthy.	Zero extra traffic; but a few real users hit the failure first.

Production systems usually run **both**: active checks for speed, passive observation as a safety net.

DESIGN YOUR `/HEALTH` ENDPOINT WELL
A good health check verifies the server can actually do its job — e.g. it can reach the database — not just that the process is running. A server that's "up" but can't reach its database should report *unhealthy* so the balancer routes around it.

6 Sticky sessions & the stateless upgrade

Section 4.3 hinted at a real tension. IP Hash exists mainly because some servers keep user data *on that specific server* (a "stateful" design). But binding users to one machine is fragile — if that server dies, those users lose their session.



The modern fix: pull session state out of the servers into a shared store, so every server is interchangeable.

RULE OF THUMB

Prefer **stateless servers** + a shared session store (like Redis) over sticky sessions. Then any algorithm works, any server can handle any user, and losing a server costs nobody their cart. Reach for IP Hash / sticky sessions only when you can't easily externalize the state.

7 Going deeper — L4 vs L7 load balancing

Load balancers come in two broad kinds, named after the network layer they operate at. This distinction shows up constantly in real design discussions and interviews.

Layer 4 (Transport)

Routes by IP + port only.

Doesn't read the actual content.

Very fast, very simple.

"Send this connection to some server."

Layer 7 (Application)

Reads the HTTP request:

URL, headers, cookies.

Smart routing, more work.

"/api → API servers, /img → image servers."

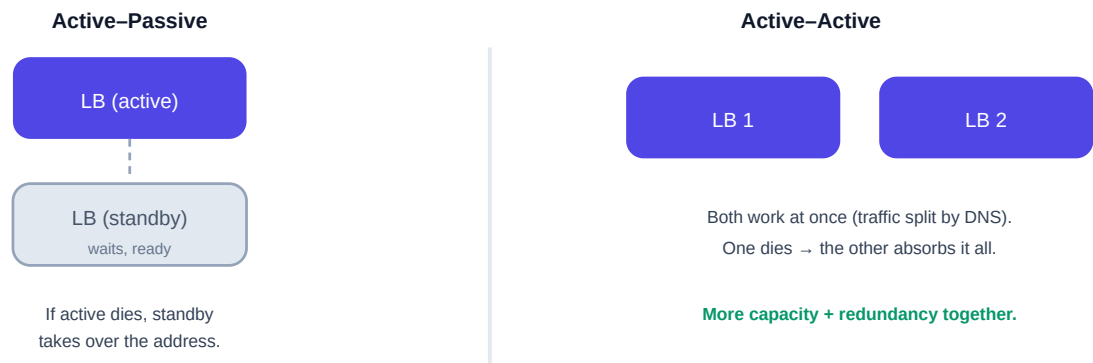
Capability	Layer 4	Layer 7
Decides based on	IP address & port	Full request: path, headers, cookies
Route <code>/api</code> vs <code>/images</code> to different pools	No	Yes
SSL/TLS termination	Usually no	Yes
Speed / overhead	Fastest	Slightly heavier
Typical tool	AWS NLB, HAProxy (TCP mode)	Nginx, AWS ALB, Envoy

ONE-LINE TAKEAWAY

L4 is a fast, content-blind traffic switch. **L7** understands HTTP and can route intelligently by URL, cookie, or header — at a small performance cost. Most web apps use L7.

8 Don't let the balancer become the single point of failure

Here's the catch you should spot immediately: we put a load balancer in front to remove single points of failure among servers... but now *everything* flows through the balancer. If it dies, the whole site is down. So the balancer itself must be made redundant.



Two common patterns for making the load-balancing layer itself highly available.

- **Active-Passive:** a second balancer sits idle, monitoring the first. If the active one fails, the standby instantly takes over its address (failover). Simple, but half your balancer capacity is idle.
- **Active-Active:** two (or more) balancers all handle traffic simultaneously, usually with DNS spreading users across them. You get extra capacity *and* redundancy — if one dies, the others carry the load.

THE INTERVIEW TRAP

Whenever you add a component "to improve reliability," ask: *is this new component now a single point of failure?* A design isn't highly available until **every** layer — servers, balancers, databases — has redundancy.

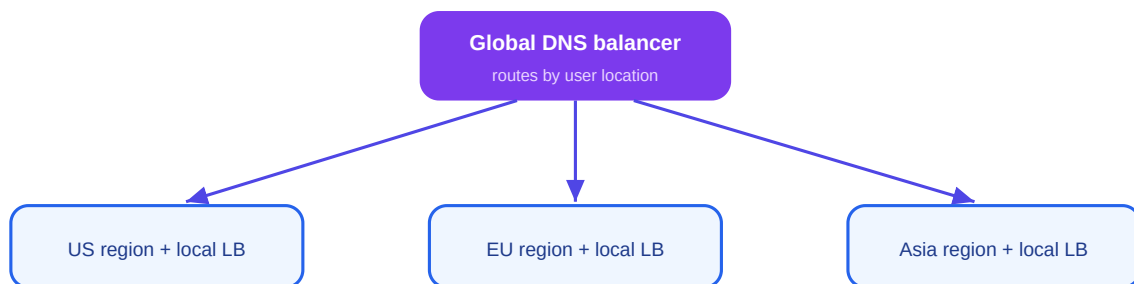
9 Advanced extras

9.1 SSL / TLS termination

Encrypting and decrypting HTTPS traffic costs CPU. A common trick is to let the **load balancer handle all the HTTPS work** — it decrypts incoming traffic, then forwards plain (fast) requests to servers over the private network. This is called *SSL termination*. Your servers do less crypto work, and you manage certificates in one place.

9.2 Global load balancing (GSLB / DNS-based)

Everything so far balances within one data center. But what if you have data centers in Europe, the US, and Asia? **Global load balancing** uses DNS to send each user to the *nearest or healthiest* region, and only then does a local balancer pick a server inside that region.



Two tiers: global DNS picks the region, a local balancer picks the server. This cuts latency and survives a whole region going down.

10 Real-world tools & how you actually use them

Tool	Type	What it's known for
Nginx	L7 (software)	Extremely popular web server that doubles as an L7 balancer; simple config.
HAProxy	L4 & L7	Battle-tested, high-performance software balancer.
AWS ALB	L7 (managed)	Application Load Balancer — routes by URL/host, integrates with auto-scaling.
AWS NLB	L4 (managed)	Network Load Balancer — ultra-low latency, millions of connections.
Envoy	L7 (software)	Modern proxy, the backbone of many service meshes.
Cloudflare / GSLB	Global	DNS-level global balancing + DDoS protection at the edge.

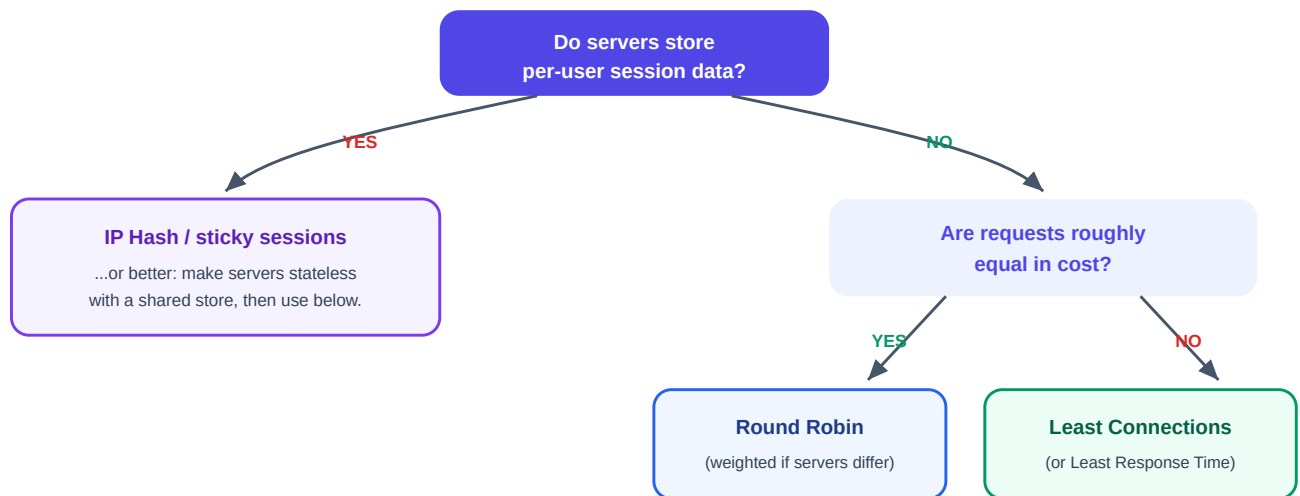
A tiny taste of config (Nginx, L7, Round Robin + health checks)

```
# Define the pool of backend servers
upstream my_app {
    server 10.0.0.11:8080;           # Server 1
    server 10.0.0.12:8080;           # Server 2
    server 10.0.0.13:8080 weight=2; # Server 3 gets 2x traffic
    # swap the line below to change the algorithm:
    # least_conn;                   # -> Least Connections
    # ip_hash;                      # -> IP Hash (sticky)
}

server {
    listen 80;
    location / {
        proxy_pass http://my_app;   # hand off to the pool
    }
}
```

That's the whole idea in ~10 lines: list your servers, pick an algorithm, point traffic at the pool. Managed cloud balancers (ALB/NLB) do the same thing through a console or a few API calls instead of a config file.

11 Decision guide — which algorithm, when?



*A quick mental flowchart for choosing. And always layer **health checks** on top — they're not optional.*

12 Cheat sheet & practice

One-glance recap

Concept	The one thing to remember
Load balancer	Traffic director in front of your servers; clients only see it.
Round Robin	Cycle through servers in order. Simple; ignores current load.
Least Connections	Send to the least-busy server right now. Best for uneven work.
IP Hash	Same user → same server. Needed for local session data.
Health checks	Auto-remove dead servers; auto-add recovered ones.
Stateless + shared store	The clean alternative to sticky sessions.
L4 vs L7	L4 = fast IP/port switch. L7 = smart HTTP-aware routing.
Redundant balancers	The balancer itself must not be a single point of failure.

Test yourself

QUESTION 1

Your app streams large video uploads — some requests take 2 seconds, others take 5 minutes. Round Robin is causing a few servers to choke while others idle. Which algorithm fixes this, and *why*?

QUESTION 2

You switch from IP Hash to Least Connections and suddenly users are getting logged out at random. What does that tell you about how your servers store sessions — and what's the better long-term fix?

QUESTION 3

A friend says "we added a load balancer, so now our site can never go down." What single question would you ask to test that claim?

WHERE THIS FITS IN THE SERIES

Day 2 gave you *many servers* (horizontal scaling). This lesson gave you the *traffic director* that makes those servers useful. A natural next step is **caching** — how to avoid doing the same work twice — and then **databases & replication**, where load balancing ideas return in a new form.